

# Master 2 SD 2023/2024 : HPC

## Examen du 06 décembre 2023

Durée : 1 heure

Support et notes de cours autorisés. Accès internet autorisé uniquement dans le cadre de la consultation des notes de cours. Le barème et les durées sont donnés à titre indicatif.

Attention il y a 4 pages

### Questions de cours (8 points) (20 minutes)

Les questions sont indépendantes.

#### Question 1 (2 points)

Un code parallèle s'exécute en 27h sur 1 cœur de calcul et en 20 minutes sur 100 cœurs de calcul. Donnez son accélération (speedup)  $S_{100}$  et son efficacité (efficiency)  $E_{100}$ .

#### Question 2 (2 points)

Pour chacun des codes suivants, indiquez quelles présentations mémoire (RMO, CMO) sont adaptées à l'imbrication des boucles.

```
#tab1 et tab2 sont deux
#tableaux à deux dimensions
for i in range(xdim):
    for j in range(ydim):
        x = tab1[i,j]
        y = tab2[i,j]
        res = (x*x+x*y+(y+x)*y)\
              *(y*y+y*x+(x+y)*x)
```

Code 1

```
#tab1 et tab2 sont deux
#tableaux à deux dimensions
for j in range(ydim):
    for i in range(xdim):
        x = tab1[i,j]
        y = tab2[i,j]
        res = (x*x+x*y+(y+x)*y)\
              *(y*y+y*x+(x+y)*x)
```

Code 2

```
#tab1 et tab2 sont deux
#tableaux à une dimension
for j in range(ydim):
    for i in range(xdim):
        x = tab1[i]
        y = tab2[j]
        res = (x*x+x*y+(y+x)*y)\
              *(y*y+y*x+(x+y)*x)
```

Code 3

#### Question 3 (2 points)

Un code multiplie terme à terme deux tableaux de 600 000 000 de flottants double précision. Donner le nombre de FLOP théorique de ce code. Estimez un temps d'exécution de ce code sur une machine similaire à Illiac IV.

#### Question 4 (2 points)

Indiquez si chacun des extraits de code suivants est adapté au portage sur GPU.

```
# m liste d'entiers >= 1
f = m
for m in nombre:
    n = m // 2
    while n >= 2:
        if m % n == 0:
            f = n
            break
    n -= 1
```

Code 4

```
#tabx, taby et res sont des
#tableaux 1D de taille n
for i in range(n):
    res[i]=(taby[i])/tabx[i]
```

Code 5

```
# res tableau ndarray 2 dimensions
for i in range(res.shape[0]):
    for j in range(res.shape[1]):
        res[i,j] = res[i-1,j-1] \
                  + res[i-1,j] + res[i,j-1]
```

Code 6

## Exercices (12 points)

Pour chacun des exercices justifiez vos réponses en quelques phrases. **Attention il y a 3 exercices. et un exercice bonus si vous avez du temps**

### Exercice 1 (2 points, 5 minutes)

Pour un problème de taille donnée, on considère que 95 % d'un programme est parfaitement parallélisé.

#### Question 1.a

Quelle est le nombre minimal de cœurs de calcul nécessaires pour obtenir une accélération (speedup) de 10 ?

#### Question 1.b

Le code s'exécute en 24 heures sur 20 cœurs de calcul. Quel temps d'exécution peut-on espérer avec 40 cœurs ? Le passage à 40 cœurs vous semble-t-il pertinent ?

### Exercice 2 (5 points, 15 minutes)

Un code applique un traitement très calculatoire sur chacune des données lu dans un fichier texte.

La lecture du fichier est faite par une fonction **read\_file** qui fait appel à deux sous-fonctions **get\_description** qui lit l'entête du fichier de données et **get\_value** qui lit une donnée spécifique dans un fichier.

Les données lues sont stockées dans un tableau de type **ndarray**. La partie calculatoire est faite par la fonction **kernel** qui applique à chaque élément du tableau le traitement **compute\_wtf**, cela nécessite d'accéder à d'autres données du tableau qui sont récupérées par la fonction **get\_data**.

On réalise un profilage pour un petit fichier (96x28). Le résultat du profilage est le suivant :

| Total execution time |        |           | 0,489s          |
|----------------------|--------|-----------|-----------------|
| Name                 | ncalls | Self time | cumulative time |
| doall                | 1      | 0.000s    | 0.489s          |
| get_value            | 2688   | 0.059s    | 0.453s          |
| get_description      | 2689   | 0.015s    | 0.156s          |
| read_file            | 1      | 0.009s    | 0.463s          |
| get_data             | 2444   | 0.006s    | 0.008s          |
| compute_wtf          | 2444   | 0.011s    | 0.016s          |
| kernel               | 1      | 0.002s    | 0.026s          |

#### Question 2.a

Quelle est la fonction la plus coûteuse en temps ?

#### Question 2.b

Selon vous le code est-il IO-Bound, Memory bound ou CPU bound ? Expliquez en quelques phrases.

#### Question 2.c

Quelle(s) fonction(s) faut-il optimiser pour améliorer les performances du codes ?

### Exercice 3 Calcul de Pi sur GPU (3 points, 7 minutes)

La valeur du nombre  $\pi$  peut être approximée par la formule suivante :

$$\pi = \int_0^1 \frac{4}{1+x^2} dx \approx \sum_{i=1}^n \frac{1}{n} \frac{4}{1+((i-1/2)/n)^2}$$

Il en découle que  $\pi$  peut être calculé par la fonction **aproxpi** (cf. Code 7)

#### Question 3.a

En quelques phrases et/ou schémas, expliquez comment cet algorithme peut être porté sur GPU (le code n'est pas demandé).

```
def aproxpi(n):  
    pi = 0  
    for i in range(1,n):  
        pi += 1. / (1. + ((i - 1. / 2.) / n)**2)  
    return 4 * pi / n
```

Code 7: Approximation de Pi

### Exercice 4 (5 points, 15 minutes)

Les 3 questions peuvent être traitées indépendamment. Les codes sont à la fin de l'énoncé.

Quentin a écrit un code qui traite des images représentant des formes géométriques noires sur fond blanc.

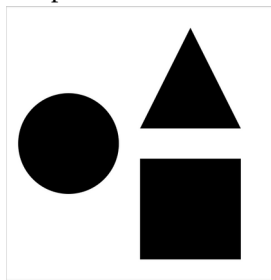


Illustration 1: Image noir et blanc

Le traitement mis en place calcule l'aire de la surface des formes géométriques.

#### Question 4.a

Pour calculer l'aire totale des surfaces géométriques Quentin décide de compter le nombre total de pixels noirs. La fonction **surface** (cf. Code 8) réalise ce travail.

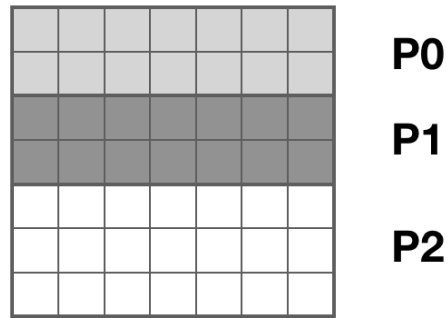
En quelques phrases et/ou schéma, proposez une méthode de parallélisation de ce traitement.

#### Question 4.b

Quentin a optimisé son algorithme de contour mais celui-ci reste très long par rapport au calcul de l'aire. Il décide de paralléliser la fonction **contour** en découpant son image suivant la dimension **xdim** :

pour **nproc** processus la dimension locale traitée est la suivante : **xdim/nproc** pour tous le processus. Comme cette division ne tombe pas nécessairement juste, le dernier processus traite les **xdim-(nproc-1)\*(xdim/nproc)** dernières lignes.

Par exemple, pour 3 processus et une image de 7 lignes on aurait,  $7//3=2$  lignes pour P0 et P1 et  $7-(2*(7//3))=3$  lignes pour P2 (voir Illustration 2).



*Illustration 2: équilibrage charge*

Pour une image de 256 lignes et 100 processus il obtient une accélération d'à peine 4,5. Pourquoi ? Proposez une meilleure solution de répartition de la charge.

Codes

```
def surface(image):
    #Retourne aire le nombre total de pixel noir.

    xdim,ydim=np.shape(image)
    #Initialement aire vaut 0
    aire=0
    for jrange in range(ydim):
        for irange in range(xdim):
            if image[irange,jrange]==black : aire+=1
    return aire
```

*Code 8: Fonction surface*

### Exercice bonus

Vipaldek a un algorithme quadratique, i.e, pour une donnée de taille  $n$  la complexité de calcul est de l'ordre de  $n^2$ . Ainsi pour une donnée de taille 100 le temps d'exécution en séquentiel est de 10000 secondes. Vipaldek suppose qu'en distribuant les données sur 100 processus chaque processus calculera en  $1^2 = 1$  seconde et donc l'accélération sera de 10000.

*Ce raisonnement vous semble-t-il justifié ? Expliquez.*